NOTE The preceding snippet uses Hamcrest, the other major Java fluent assertion library.

This isn't a very clean solution. The page object isn't only reporting the state of the page; it's also making assertions about it, which should be the responsibility of the test logic. A better solution would be to have the page object return the state of the Search button, and let the test do the asserting:

```
public class BookingPage {
    ...
    WebElement search;

    public boolean searchButtonIsEnabled() {
       return search.isEnabled();
    }
}
```

Just return the state of the Search button.

Now the test code will be able to decide what assertion is the most appropriate:

```
assertThat(bookingPage.searchButtonIsEnabled(), is(true));
```

This approach provides a better separation of concerns and avoids the risk of having bloated page object classes with assertion methods that are only used by a single test.

There's still one area where you could improve this example. Any test that simply asserts that something is true (or false) runs the risk of providing poor diagnostic information. For example, both of the assertions in the preceding examples will report the following error message if they fail:

```
Expected: is <true>
     but: was <false>
```

When a test fails, it's important for diagnostic messages to be informative, and most assertion libraries let you add customized messages. For example, you could rewrite the earlier test to provide a more informative message, as shown here:

```
assertThat("Search button should be enabled",
           bookingPage.searchButtonIsEnabled());
```

There's still a little duplication in this code. Some WebDriver-based libraries also provide extensions that can produce more useful assert messages with even less effort. For example, in Thucydides, you can use the `WebElementState` class to return the state of a web element to the test. You could use a Thucydides page object for this page like this:

```
public class BookingPage extends PageObject {

    WebElement search;
       ...
    public WebElementState searchButton() {
        return $(search);
    }
}
```

The page object extends the Thucydides PageObject base class.

The $() method returns information about the current state of the web element.